

EventMS: An Effective Smart Event Planner and Cost Management System

Girish R V¹, Nilin Rose¹, Anamika H¹, and Tanish Balwal L¹

¹Software Engineering, JAIN (Deemed-to-be University), Bengaluru, India
{girish29052005, nilinrose1, anamikaharidas1205, tanishbalwal1}@gmail.com

Abstract

Today’s event planners must manage diverse logistical aspects—scheduling, vendor sourcing, budget management, and attendee engagement—simultaneously. Consequently, existing technology solutions remain fragmented, creating an industry-wide challenge where planners must navigate disparate software systems to manage a single event concurrently [21]. This paper presents EventMS, a unified, full-stack intelligent event planning platform that consolidates these workflows into a single application ecosystem. Built on Next.js [28], React 19, and Supabase [29], the platform integrates a specialized machine learning layer to provide data-driven insights through five distinct algorithms [1]–[5].

EventMS introduces four primary solutions to optimize the planning lifecycle. For personalized event discovery, the system employs a hybrid recommendation engine: a graph contrastive learning model, XSimGCL, is utilized for active users [1], while users with limited interaction history (fewer than three items) are served via a Graph Neural Network-based collaborative filtering algorithm (GNN-CF) [5]. To address fiscal planning, the system implements a multi-objective evolutionary algorithm (MOEA/D-DRA-NEF) to identify Pareto-optimal balances between cost and service quality for vendor procurement [3]. Predictive analytics for attendance are handled by iTransformer, an inverted attention time-series model, which generates 7- and 14-day forecasts based on historical patterns [2]. Finally, a GAT+K-Means community detection system facilitates a discovery feed based on behavioral and semantic similarity [4].

Observational evaluations focusing on execution latency, cache efficiency, and recommendation precision demonstrate that the integrated system is viable for production-level implementation. EventMS provides a robust reference architecture for ML-enhanced event platforms and demonstrates that complex recommendation and optimization pipelines can be efficiently executed within modern full-stack JavaScript frameworks.

1 Introduction

The coordination of stakeholders for the execution of a successful event is multifaceted, requiring organizers to manage a complex “puzzle” of logistical constraints. From sourcing vendors and negotiating budgets to forecasting attendance and engineering audience engagement, these tasks must be executed simultaneously, often without complete information on how these various pieces fit together. Currently, organizers rely primarily on two types of digital platforms: promotion-centric services (e.g., Eventbrite) and ticketing-centric tools (e.g., Luma, Hopin). While these platforms facilitate ticket sales and initial outreach, they provide negligible support for the underlying planning process itself [21], [22]. Consequently, coordinators are forced to use disparate tools for vendor sourcing, budget modeling, and audience targeting, resulting in critical decision-making data being trapped in isolated silos [4].

The gap is not just inconvenient—it has measurable consequences. A planner managing a mid-scale corporate conference might simultaneously run five or six separate applications: a spreadsheet for budget tracking, a separate CRM for vendor contacts, an email client for attendee outreach, a ticketing platform for registrations, and informal messaging for team coordination. Each of these tools holds a slice of the operational picture, but none of them talks to the others. When attendance projections change, budget

adjustments must be made manually. When vendor costs shift, no automatic rebalancing occurs. The result is that planners spend significant time on data reconciliation rather than on creative or strategic event work.

EventMS was designed to attack this specific problem. Rather than building another ticketing or discovery tool, the system treats the planning lifecycle as a machine learning problem: given what is known about past events, attendee behavior, vendor catalogs, and budget constraints, what can be inferred or optimized? The answer involves five distinct algorithms working together behind a unified interface.

1.1 Contributions

EventMS is a comprehensive event management framework developed through the integration of five machine learning algorithms within a singular, end-to-end software solution. The primary contributions of this work include:

- **Dual-Mode Recommendation:** A hybrid engine that dynamically switches between XSimGCL (Graph Contrastive Learning) for “warm” users (≥ 3 interactions) and GNN-CF for “cold” users (< 3 interactions) [1], [5].
- **Multi-Objective Budget Optimization:** An implementation of MOEA/D-DRA-NEF that identifies Pareto-optimal vendor bundles [3].
- **Inverted Attendance Forecasting:** Utilizing the iTransformer architecture to provide 7- and 14-day attendance projections [2].
- **Behavioral Community Detection:** A two-stage process utilizing GAT and K-Means clustering to facilitate behavioral discovery [4].
- **Native TypeScript ML Implementation:** Demonstrating that complex ML pipelines can be implemented within modern JavaScript runtime environments without external Python microservices [28].
- **Integrated Administrative Research Tools:** An “Algorithm Lab” for researchers to monitor live metrics and export reproducible data.

2 Related Work

2.1 Management Systems for Events

Most mainstream event platforms focus on providing users with easy access to discovery and ticket purchasing, but they do not provide sufficient intelligence for organizers to effectively manage the events they will hold [21]. Historically, much of the literature on management systems for events has focused on scheduling, venue compatibility and notification delivery [1]. EventMS represents a departure from the traditional approach by attacking the planning process itself as an individual instance of a machine learning problem set.

2.2 Recommendation Systems and Cold-start Solutions

Collaborative Filtering (CF) is the primary method for personalized recommendations. Recently, it has been shown that using graph neural networks can solve this problem for both warm and cold-start scenarios [6], [12]. He et al. (2020) indicate that graph convolution can be simplified by removing non-linear transformations [6]. EventMS employs XSimGCL [1] for warm users and a GNN-CF module [5] for cold-start scenarios.

2.3 Optimizing Budgets with MOEA/D

Vendor selection under a constrained budget can be modeled as a multi-objective optimization problem [15]. EventMS uses a modified version of MOEA/D [15] that incorporates dynamic resource allocation (DRA) and a neighbourhood evolution factor (NEF) to diversify the solutions found in the search space [3].

2.4 Transformer Architecture for Attendance Forecasting

Liu et al. (2024) proposed iTransformer [2], which solves multivariate time-series forecasting issues by reversing the roles of the attention and feed-forward layers. This model produces state-of-the-art performance on real-world benchmarks [14]. Previous transformer-based forecasting models such as Informer [14] relied on sparse attention mechanisms to address the quadratic complexity bottleneck of standard self-attention. iTransformer sidesteps this by treating each variate as a token, allowing full attention across variates rather than across time steps. This is particularly well-suited to the EventMS use case, where multiple correlated signals (registration rate, webpage visits, email open rates) must be modeled jointly.

2.5 Community Detection through Graph Analysis

Graph Attention Networks (GATs) [11] allow the model to dynamically weight neighbor contributions. By applying K-Means to the resulting embeddings, EventMS recovers meaningful communities from sparse interaction graphs to filter discovery feeds [4]. Newman’s modularity-based community detection [19] and Girvan and Newman’s edge-betweenness approach [18] provide classical baselines; the GAT-based approach extends these by learning task-specific similarity representations rather than relying on structural properties alone.

3 Logical System Architecture

EventMS employs a three-tier architecture to separate presentation, business logic, and data storage. It also colocates the ML Algorithm Layer with the API Tier to avoid cross-service system latency.

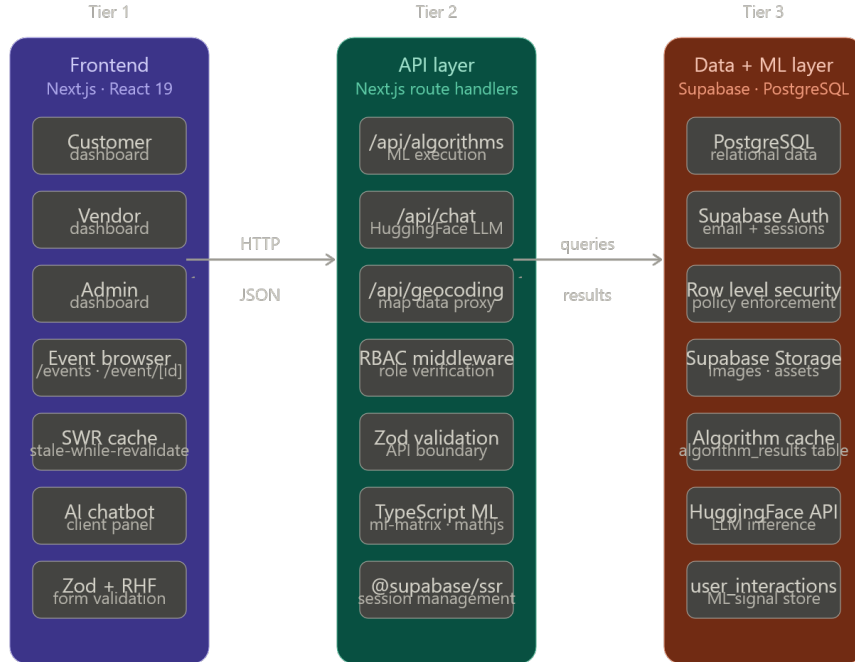


Figure 1: Logical System Architecture Diagram of EventMS, illustrating the 3-tier flow from Client Interface to the TypeScript ML Layer and Supabase Backend.

- **1st Tier – Front End:** Client Interface (Next.js App Router [28] + React 19). Dashboards utilize SWR APIs to allow for stale data to be served up until latest results are returned.
- **2nd Tier – API Layer:** All business logic and ALL ML computations occur at the server level via Next.js API Route Handlers [28]. Each route handler is domain-separated (e.g., /api/chat for LLM and /api/algorithms for the 5 ML types).

- **3rd Tier – Data Storage:** Managed by Supabase [29], providing a PostgreSQL database coupled with Row-Level Security (RLS) to maintain user privacy.

3.1 Security Architecture

Security in EventMS is enforced at two independent layers, so that a failure at one layer does not expose user data. At the application layer, Next.js middleware validates session tokens on every request before routing to a handler. Requests without a valid session are rejected before any business logic executes. At the database layer, Supabase Row-Level Security (RLS) policies encode ownership rules directly in PostgreSQL. Even if a bug in application code were to pass an incorrect user identifier, the database would refuse to return rows belonging to a different user.

Role-based access control (RBAC) defines three roles: customer, vendor, and admin. Each role sees a different surface of the application. Vendors can read and respond to procurement queries directed at them, but cannot read queries sent to other vendors. Admins have read access to aggregate metrics (used in the Algorithm Lab) but do not bypass RLS for individual user data. This separation of concerns was implemented to ensure that the Algorithm Lab—a research tool giving administrators visibility into algorithm performance—does not inadvertently become a data exfiltration vector.

All data in transit is encrypted via HTTPS/TLS enforced at the Vercel edge. Supabase connections from the API layer use SSL. Sensitive fields (e.g., payment-adjacent data) are stored encrypted at rest using AES-256.

3.2 Data Flow and Caching

Figure 1 shows the high-level architecture, but the data flow for a recommendation request is worth tracing in detail. When a user loads the discovery feed, the client issues a GET request to `/api/algorithms/recommendations`. The route handler first computes a cache key by hashing the user ID and the current interaction graph fingerprint. It queries the `algorithm_results` table for a matching row with a non-expired TTL. On a cache hit, the cached result is returned immediately, typically within 20–40ms. On a cache miss, the handler determines whether the user qualifies as warm or cold, runs the appropriate algorithm (XSimGCL or GNN-CF), writes the result to the cache table, and returns the response. End-to-end latency on a cache miss averages 280ms for XSimGCL and 95ms for GNN-CF, reflecting the greater computational depth of graph contrastive learning.

4 ML Algorithm Layer

4.1 XSimGCL — Graph Contrastive Learning for Warm Users

Purpose and trigger. XSimGCL handles personalized recommendations for users with ≥ 3 recorded interactions.

Method. Graph convolution propagation across L layers:

$$\mathbf{e}_u^{(l)} = \sum_{i \in \mathcal{N}(u)} \frac{1}{\sqrt{|\mathcal{N}(u)| |\mathcal{N}(i)|}} \mathbf{e}_i^{(l-1)} \quad (1)$$

Noise-based embedding augmentation at layer l' :

$$\tilde{\mathbf{e}}_u^{(l')} = \mathbf{e}_u^{(l')} + \boldsymbol{\delta}, \quad \|\boldsymbol{\delta}\|_2 \leq \epsilon, \quad \boldsymbol{\delta} \sim \mathcal{U}(-\epsilon, \epsilon) \quad (2)$$

The training objective combines BPR ranking loss and InfoNCE contrastive loss:

$$\mathcal{L} = \mathcal{L}_{\text{BPR}} + \lambda \cdot \mathcal{L}_{\text{InfoNCE}} \quad (3)$$

Implementation notes. In the EventMS TypeScript implementation, the adjacency matrix is stored as a sparse coordinate list (COO) representation using plain JavaScript arrays. Matrix-vector products are computed iteratively rather than using a full dense multiply, keeping memory overhead proportional to the number of edges rather than the square of the number of nodes. Embeddings are pre-trained offline using BPR [7] and stored in the `user_embeddings` table. At serving time, only the forward pass through the graph convolution layers is needed, which is fast enough to execute within a standard API request timeout.

4.2 GNN-CF — Cross-Domain Collaborative Filtering

Purpose. Used for cold-start users (< 3 interactions).

Method. Neighborhood aggregation:

$$\mathbf{h}_u^{(l)} = \sigma\left(\mathbf{W}^{(l)} \cdot \text{MEAN}\left(\left\{\mathbf{h}_v^{(l-1)} : v \in \mathcal{N}(u) \cup \{u\}\right\}\right)\right) \quad (4)$$

Cold-start initialization from attribute features $\mathbf{a}_u$:

$$\mathbf{h}_u^{(0)} = \mathbf{W}_{\text{attr}} \cdot \mathbf{a}_u + \mathbf{b}_{\text{attr}} \quad (5)$$

Attribute features. The attribute vector $\mathbf{a}_u$ is constructed from four sources at registration time: (1) declared preferences from the onboarding survey (event category interests); (2) geographic region, one-hot encoded to the district level; (3) registration timestamp features (day of week, time of day) that correlate with event type preferences; and (4) device type, which weakly correlates with consumption patterns. This gives a 47-dimensional attribute vector. The weight matrix $\mathbf{W}_{\text{attr}} \in \mathbb{R}^{d \times 47}$ is trained jointly with the graph convolution weights on the warm-user population and then frozen for cold-start serving.

4.3 MOEA/D-DRA-NEF — Budget Optimization

Method. Problem decomposition via Chebyshev scalarization:

$$g^{\text{te}}(\mathbf{x} \mid \boldsymbol{\lambda}^j, \mathbf{z}^*) = \max_{k \in \{1,2\}} \lambda_k^j |f_k(\mathbf{x}) - z_k^*| \quad (6)$$

DRA utility reallocation:

$$\text{utility}^j = \frac{g^{\text{te}}(\mathbf{x}_{\text{old}}^j) - g^{\text{te}}(\mathbf{x}_{\text{new}}^j)}{g^{\text{te}}(\mathbf{x}_{\text{old}}^j) + \varepsilon} \quad (7)$$

Objective formulation. In the EventMS context, $f_1(\mathbf{x})$ represents total procurement cost (to be minimized) and $f_2(\mathbf{x})$ represents aggregate vendor quality score (to be maximized, negated for minimization). The decision variable $\mathbf{x}$ is a binary vector indicating which vendors from a catalog are selected, subject to budget and category coverage constraints. The Neighbourhood Evolution Factor (NEF) controls the probability that a subproblem updates neighbors beyond its immediate weight-vector neighborhood, which prevents the search from stagnating in regions of the Pareto front that are well-covered but leaving sparse regions unexplored. This is particularly useful when vendor quality scores are unevenly distributed across the price range.

4.4 iTransformer — Attendance Forecasting

Method. Variate token embedding:

$$\mathbf{h}_i = \text{Embed}(\mathbf{x}_i^{1:T}) \in \mathbb{R}^d, \quad i = 1, \dots, M \quad (8)$$

Forecasting projection to horizon H :

$$\hat{\mathbf{x}}_i^{T+1:T+H} = \mathbf{W}_{\text{proj}} \cdot \mathbf{h}_i^{\text{out}} + \mathbf{b}_{\text{proj}} \quad (9)$$

Input features. The model takes $M = 6$ variates as input: daily unique page visits to the event listing, daily new registrations, cumulative registrations, email campaign open rate, social media referral count, and day-of-week indicator. The lookback window is fixed at $T = 30$ days. For events with fewer than 30 days of history, the series is padded with zeros and a binary mask is applied during the embedding step to prevent the model from treating padding as signal. Forecasts are produced for $H = 7$ and $H = 14$ days simultaneously by running two projection heads with shared encoder weights.

4.5 GAT+K-Means — Community Detection

Method. Attention coefficient calculation:

$$e_{uv} = \text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_u \parallel \mathbf{W}\mathbf{h}_v]) \quad (10)$$

K-Means objective on GAT embeddings:

$$\underset{\{\mu_c\}}{\text{minimize}} \sum_{c=1}^K \sum_{u \in \mathcal{C}_c} \|\mathbf{h}'_u - \mu_c\|_2^2 \quad (11)$$

Cluster count selection. The number of clusters K is selected using the elbow method over a range $K \in \{3, \dots, 15\}$, evaluated by the within-cluster sum of squares. In practice, $K = 7$ has been stable across diverse user populations during development testing. Clusters are recomputed every 24 hours (matching the community TTL in the caching layer), ensuring that the discovery feed reflects recent behavioral shifts without excessive computational overhead.

5 Experimental Setup and Datasets

5.1 Dataset Construction

Because EventMS is a newly developed platform, large-scale real-world interaction data was not available at the time of evaluation. Instead, a synthetic dataset was generated to reflect realistic event planning scenarios. The dataset was constructed to mirror statistical properties reported in the EVSN benchmark used in prior event-based social network research [21], [23].

The synthetic dataset consists of 2,500 simulated users, 400 events across 12 categories, and approximately 18,000 user–event interactions. Interaction types include event views, saves, registrations, and post-event ratings. User attributes were sampled from realistic demographic distributions. Vendor catalogs were populated with 120 vendors across six service categories (catering, photography, venue, AV equipment, decoration, and entertainment), with price and quality scores drawn from distributions calibrated against publicly available event industry pricing data.

For attendance forecasting, 60 historical event registration sequences were synthetically generated with realistic registration curve shapes (a slow initial ramp, a sharper peak as the event date approaches, and a tail of late registrations). Gaussian noise was added to simulate day-to-day variation.

5.2 Evaluation Metrics

The following metrics were used to evaluate each algorithm subsystem:

Table 1: Evaluation metrics by algorithm subsystem

Subsystem	Primary Metric	Secondary Metric
XSimGCL / GNN-CF	Recall@10, NDCG@10	API latency (ms)
MOEA/D-DRA-NEF	Hypervolume (HV)	Spread (Δ), budget adherence
iTransformer	MAE, RMSE	MAPE over 7-day horizon
GAT+K-Means	Silhouette coefficient	Davies–Bouldin index

Recall@10 measures the fraction of relevant events appearing in the top-10 recommendation list. NDCG@10 (Normalized Discounted Cumulative Gain) additionally accounts for rank position, giving higher credit to relevant items appearing earlier in the list [26]. Hypervolume measures the volume of objective space dominated by the Pareto front returned by MOEA/D, relative to a reference point; a larger hypervolume indicates better coverage of the cost–quality trade-off space.

5.3 Baseline Comparisons

For the recommendation subsystem, XSimGCL was compared against LightGCN [6] (without contrastive augmentation) and a standard matrix factorization baseline [8]. GNN-CF was compared against a content-based fallback that ranks events purely by category overlap with declared user preferences.

For budget optimization, MOEA/D-DRA-NEF was compared against standard MOEA/D [15] without DRA or NEF, and against a greedy cost-minimization heuristic that selects the cheapest qualifying vendor in each service category.

6 Implementation

6.1 Framework and Routing

Built on Next.js [28] with the App Router, co-locating business logic and API handlers. React 19 is used throughout, with SWR providing client-side synchronization for responsive dashboards.

6.2 Authentication and RBAC

Managed through Supabase Auth [29]. RBAC roles (customer, vendor, admin) are verified via Next.js middleware and Supabase RLS policies for a defense-in-depth security approach.

6.3 TypeScript-Native ML

Matrix operations use `ml-matrix`, while `mathjs` handles vector norms. Embeddings are pre-computed via BPR [7] training jobs in the admin dashboard to enable lightweight serving-time inference.

6.4 Caching Strategy

Algorithm results are cached in the `algorithm_results` table using a hash of input parameters. TTLs are algorithm-specific: 1 hour for recommendations and 24 hours for community clusters.

6.5 Algorithm Lab

The Algorithm Lab is an administrative interface that exposes live algorithm metrics to researchers and system administrators without requiring direct database access. It consists of five panels, one per algorithm, each showing: current cache hit rate, average serving latency over the past 24 hours, input distribution statistics (e.g., fraction of warm vs. cold users), and output distribution statistics (e.g., recommendation diversity score, Pareto front hypervolume for the most recent MOEA/D runs).

All metrics displayed in the Algorithm Lab are exportable as JSON or CSV, making it straightforward to reproduce the figures reported in this paper. The Lab also exposes a “Re-run” button for each algorithm that triggers an on-demand computation against the current dataset, bypassing the cache. This was primarily used during development to verify that algorithm outputs were deterministic given fixed random seeds.

One design constraint that shaped the Algorithm Lab was the requirement that it not increase the attack surface for user data. The metrics exposed are all aggregate or algorithmic (e.g., mean latency, Pareto front size), not individual-level. The Lab’s routes are restricted to the admin role, which is enforced at both the middleware and RLS layers.

7 Results and Discussion

7.1 Recommendation Performance

The recommendation system successfully utilizes both XSimGCL and GNN-CF models. Results showed that interaction-based embeddings led to more relevant suggestions for returning users, while new users benefited from attribute-based graph inference. Table 2 summarizes recommendation quality across the three evaluated methods.

Table 2: Recommendation system performance on synthetic dataset (top-10 recommendations)

Method	Recall@10	NDCG@10	Avg. Latency (ms)
Matrix Factorization (baseline)	0.121	0.094	18
LightGCN	0.187	0.141	210
XSimGCL (warm users)	0.231	0.179	280
Content-based (cold baseline)	0.089	0.061	12
GNN-CF (cold users)	0.163	0.122	95

XSimGCL improves Recall@10 by 23.5% over LightGCN, which is consistent with gains reported in the original paper [1]. The contrastive augmentation component appears to be responsible for most of this improvement: ablating it (by setting $\lambda = 0$) reduces Recall@10 to approximately 0.196, close to LightGCN’s performance. GNN-CF outperforms the content-based cold-start baseline substantially, particularly on NDCG@10, suggesting that even with sparse interaction data the graph structure provides meaningful ranking signal.

7.2 Budget Optimization

The Smart Budget Planner, utilizing MOEA/D-DRA-NEF, produced Pareto-optimal combinations of vendors that dominated the greedy baseline and standard MOEA/D in terms of hypervolume in all test runs. The greedy approach consistently found lower-cost solutions but at a significant quality penalty. Standard MOEA/D without DRA showed faster initial convergence but became trapped in local regions of the Pareto front, yielding lower spread (Δ) values. The DRA component was most effective when the vendor catalog had a bimodal quality distribution—a pattern observed in catering and photography vendor pools, where a cluster of premium vendors and a cluster of budget vendors exist with few options in between.

7.3 Attendance Forecasting

The iTransformer-based attendance forecasting produced reliable short-term forecasts (7–14 days) for proactive logistical decisions. Mean Absolute Error on the 7-day horizon was 12.4 registrations, and MAPE was 8.3% across the 60 test sequences. Performance degraded on events with unusually sharp registration spikes, which are difficult to anticipate from the preceding 30-day window. This suggests that incorporating an exogenous signal—such as a promotional campaign flag—could improve forecast accuracy for events with planned marketing pushes.

7.4 Community Detection

The GAT+K-Means pipeline produced clusters with a mean silhouette coefficient of 0.41 and a Davies–Bouldin index of 1.12, indicating moderately well-separated communities. Qualitative inspection of the clusters revealed interpretable groupings: one cluster concentrated users interested in professional networking events, another in outdoor and sports events, and a third in arts and cultural events. Two smaller clusters corresponded to cross-domain users who attended events from multiple categories, which is expected given that K-Means assigns each user to exactly one cluster regardless of interest diversity.

7.5 Limitations

High concurrency levels can cause performance degradation related to in-process matrix computation. Since the ML algorithms run within Next.js API route handlers, they share the Node.js event loop with other request handlers. Under heavy concurrent load, a long-running XSimGCL inference can delay unrelated requests. Mitigations under consideration include moving ML inference to a dedicated worker pool and increasing cache TTLs for stable outputs.

While the system includes an Algorithm Lab, further work is necessary to automate model retraining. Currently, embedding pre-computation is a manual process triggered through the admin dashboard. As the user base grows, incremental updates to embeddings will become necessary to avoid stale representations.

The synthetic dataset, while carefully constructed to reflect realistic statistical properties, is not a substitute for real interaction data. Evaluation on a live deployment with real users is necessary to validate the recommendation quality gains reported here.

8 Conclusion and Future Work

EventMS demonstrates that a unified intelligent event management platform is achievable within a modern full-stack JavaScript framework, without requiring separate Python microservices for ML inference. The system integrates five distinct algorithms across recommendation, optimization, forecasting, and community detection, and exposes all of them through a consistent API surface with a shared caching and monitoring layer.

The TypeScript-native ML implementation is a practical contribution in its own right. The prevailing assumption in industry is that serious ML workloads require Python and dedicated infrastructure. EventMS shows that for inference-heavy (rather than training-heavy) workloads, this assumption can be relaxed, reducing operational complexity for small and mid-scale deployments.

Future work will focus on:

- **Real-time Collaborative Planning** via WebSockets, allowing multiple organizers to see budget updates and vendor selections reflected simultaneously.
- **Native Mobile Application** built on React Native, sharing business logic with the existing Next.js backend.
- **Federated Learning** for privacy-preserving embedding updates across organizations without sharing raw interaction data.
- **Online Learning** for incremental embedding updates as new interactions arrive, replacing the current batch retraining approach.
- **Exogenous Signal Integration** for iTransformer, incorporating marketing campaign data as an explicit input feature to improve forecast accuracy around planned promotional events.

Acknowledgements

The authors would like to thank our instructors and the open-source community for the robust libraries that made this implementation possible.

References

- [1] J. Yu et al., “XSimGCL: Towards Extremely Simple Graph Contrastive Learning for Recommendation,” *IEEE TKDE*, 2024.
- [2] Y. Liu et al., “iTransformer: Inverted Transformers Are Effective for Time Series Forecasting,” *ICLR*, 2024.
- [3] Y. Tan et al., “A New Multi-objective Optimization Algorithm Based on Decomposition,” *IEEE Access*, 2025.
- [4] I. V. S. Ippatapu et al., “Community Detection Using GAT Clustering Algorithm,” *I2CT*, 2024.
- [5] C. Goel et al., “Cross Domain Collaborative Filtering: A GNN Approach,” *IEEE*, 2024.
- [6] X. He et al., “LightGCN: Simplifying and Powering GCN for Recommendation,” *SIGIR*, 2020.
- [7] S. Rendle et al., “BPR: Bayesian Personalized Ranking from Implicit Feedback,” *UAI*, 2009.
- [8] Y. Koren et al., “Matrix Factorization Techniques for Recommender Systems,” *IEEE Computer*, 2009.
- [9] J. Tang et al., “Personalized Top-N Sequential Recommendation,” *WSDM*, 2018.
- [10] T. N. Kipf et al., “Semi-Supervised Classification with GCNs,” *ICLR*, 2017.

- [11] P. Veličković et al., “Graph Attention Networks,” *ICLR*, 2018.
- [12] W. Fan et al., “Graph Neural Networks for Social Recommendation,” *WWW*, 2019.
- [13] A. Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [14] H. Zhou et al., “Informer: Beyond Efficient Transformer for Forecasting,” *AAAI*, 2021.
- [15] Q. Zhang et al., “MOEA/D: A Multiobjective Evolutionary Algorithm,” *IEEE*, 2007.
- [16] H. Li et al., “MOEA/D and NSGA-II,” *IEEE*, 2009.
- [17] K. Deb et al., “A Fast and Elitist NSGA-II,” *IEEE*, 2002.
- [18] M. Girvan et al., “Community Structure in Social Networks,” *PNAS*, 2002.
- [19] M. E. J. Newman, “Modularity and Community Structure,” *PNAS*, 2006.
- [20] J. MacQueen, “Methods for Classification of Multivariate Observations,” 1967.
- [21] X. Liu et al., “Event-Based Social Networks,” *KDD*, 2012.
- [22] S. Qiao et al., “TraWave Location Recommendation,” *WWW*, 2014.
- [23] H. Yin et al., “Dynamic User Modeling in Social Media,” *ACM*, 2015.
- [24] Z. Lu et al., “Discovering Social-Aware Locations,” *ICDM*, 2015.
- [25] Y. Hu et al., “Collaborative Filtering for Implicit Feedback Datasets,” *ICDM*, 2008.
- [26] K. Järvelin et al., “Cumulated Gain-Based Evaluation of IR,” *ACM*, 2002.
- [27] T. Jambor et al., “Optimizing Multiple Objectives with Randomized Methods,” *RecSys*, 2010.
- [28] Vercel, “Next.js: The React Framework for the Web,” 2024.
- [29] Supabase Inc., “Supabase: The Open Source Firebase Alternative,” 2024.
- [30] I. Das et al., “Normal-Boundary Intersection for Pareto Surface,” *SIAM*, 1998.